

Deteksi Pemalsuan Tanda Tangan Offline

Dokumen Penjelasan: Dataset, Pemilihan Model, dan Workflow

1. Dataset yang Digunakan

Penelitian ini menggabungkan tiga sumber tanda tangan agar model tidak hanya menghafal satu dataset, melainkan belajar membedakan tanda tangan lintas gaya penulisan dan lintas kualitas gambar. Dua sumber merupakan basis data publik standar (CEDAR dan GPDS), sedangkan satu sumber dikumpulkan sendiri.

Ringkasan Jumlah Data

Dataset	Penulis	Genuine / org	Palsu / org	Total Genuine	Total Palsu
CEDAR	55	24	24	1.320	1.320
GPDS (subset)	150	8	14	1.200	2.100
Lokal (Dataset Sendiri)	16	~16	0	257	0
TOTAL	221	—	—	2.777	3.420

Catatan: hanya tanda tangan genuine yang dipakai untuk melatih model; gambar palsu hanya dipakai saat pengujian (meniru kondisi nyata: sistem didaftarkan dengan TTD asli, dan baru bertemu pemalsuan saat verifikasi).

Struktur & Rincian Tiap Dataset

a. CEDAR — 55 penulis. Setiap penulis punya 24 tanda tangan asli dan 24 tanda tangan palsu (skilled forgery), sehingga total 1.320 asli + 1.320 palsu. Disimpan sebagai file datar dengan pola nama `original_<id>_<n>.png` (asli) dan `forgeries_<id>_<n>.png` (palsu).

b. GPDS (subset) — 150 penulis. Setiap penulis punya 8 tanda tangan asli dan 14 tanda tangan palsu, sehingga total 1.200 asli + 2.100 palsu. Disimpan dalam folder per penulis, masing-masing berisi subfolder `genuine/` dan `forge/`. Karena ID penulisnya sampai 150, dataset ini memberi keragaman penulis paling besar.

c. Dataset Sendiri (lokal) — 16 penulis, 257 tanda tangan asli (rata-rata ~16 per orang), tanpa contoh palsu. Disimpan dalam folder `TTD ASLI/<penulis>/`. Dataset ini menambah variasi tanda tangan nyata di luar dua basis data publik.

Total gabungan: 221 penulis. Untuk pelatihan, jumlah gambar asli per penulis dibatasi maksimal 12 agar penulis dengan sampel banyak tidak mendominasi, sehingga terbentuk kumpulan kerja 2.052 gambar asli. Data ini dibagi di tingkat penulis: 188 penulis untuk pelatihan dan 33 penulis disisihkan penuh (held-out) untuk validasi dan pengujian — tidak ada satu penulis pun yang muncul di dua sisi.

2. Pemilihan Model

Model yang digunakan adalah **EfficientNet-B0** sebagai tulang punggung (backbone) ekstraksi fitur, digabung dengan sebuah **projection head** yang dilatih dengan pendekatan **metric learning (batch-hard triplet loss)**. Berikut alasan pemilihannya.

Alasan Pemilihan

- **Efisien namun akurat.** EfficientNet-B0 memakai teknik compound scaling sehingga mencapai akurasi setara jaringan yang jauh lebih besar (seperti ResNet-50) dengan parameter sekitar 10× lebih sedikit. Ini penting karena data tanda tangan berlabel terbatas — model yang ringkas lebih kecil risiko overfitting dan bahkan bisa dilatih tanpa GPU.
- **Metric learning, bukan klasifikasi biasa.** Alih-alih menebak 'ini tanda tangan siapa', model belajar memetakan gambar ke sebuah vektor sehingga jarak antar-vektor mencerminkan kemiripan.

Pendekatan ini bersifat writer-independent: penulis baru yang belum pernah dilihat saat pelatihan tetap bisa diverifikasi tanpa melatih ulang model.

- **Fine-tuning ringan (backbone dibekukan).** Hanya bagian head yang dilatih ulang, sementara backbone dibekukan. Ini hemat komputasi (cukup CPU), sekaligus menunjukkan seberapa besar kontribusi head secara terisolasi.

Kemampuan Model (hasil pada 33 penulis hold-out)

Performa diukur terpisah untuk dua jenis serangan, karena keduanya sangat berbeda tingkat kesulitannya:

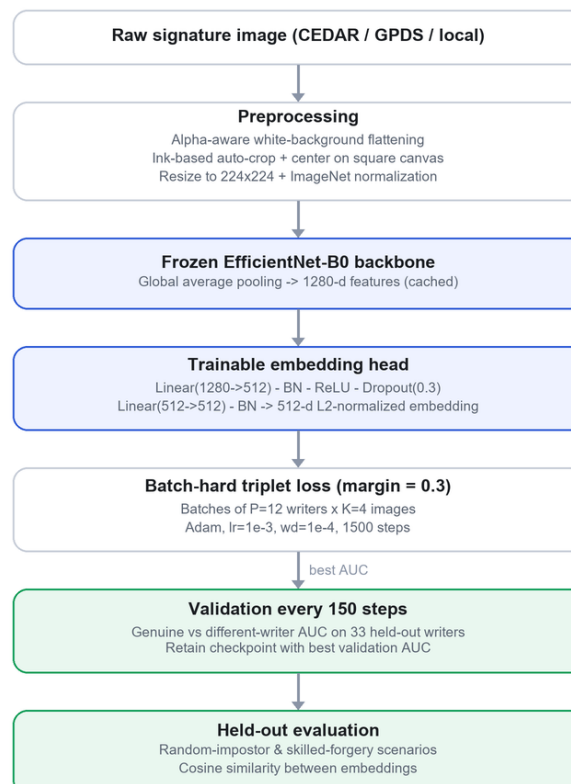
Skenario	AUC	EER	Akurasi
Random impostor (orang lain)	0,979	7,07%	90,20%
Skilled forgery (dijiplak)	0,871	20,20%	79,49%

Fine-tuning head meningkatkan seluruh metrik dibanding model dasar (baseline): pada serangan acak AUC naik dari 0,953 menjadi 0,979, dan pada pemalsuan terlatih AUC naik dari 0,817 menjadi 0,871.

Keterbatasan: pemalsuan terlatih (skilled forgery) tetap paling sulit dideteksi (EER 20,20%, hampir tiga kali lipat serangan acak), karena pemalsu sengaja meniru bentuk global tanda tangan; pembeda halus pada dinamika goresan belum sepenuhnya tertangkap oleh backbone.

3. Workflow / Alur Kerja

Diagram di bawah menunjukkan alur sistem dari gambar mentah hingga keputusan verifikasi. Setiap tahap dijelaskan setelahnya.



Gambar 1. Diagram alur sistem.

Secara garis besar, data berjalan seperti ini: **gambar bebas ukuran → 224×224 piksel → 1.280 angka → 512 angka → 1 skor kemiripan → keputusan asli/palsu**. Tabel berikut merangkum perubahan bentuk data di tiap tahap, lalu tiap tahap dijelaskan satu per satu.

Tahap	Masuk	Keluar	Ukuran data
1	File gambar tanda tangan	Citra mentah	bebas (mis. 512×256)
2	Citra mentah	Citra terstandar	3 × 224 × 224
3	Citra terstandar	Vektor ciri visual	1.280 angka
4	Vektor ciri visual	Embedding (L2-norm)	512 angka
5	Embedding satu batch	Nilai loss → perbaiki head	1 angka
6	Model saat itu	Skor AUC validasi	1 angka
7	Dua embedding	Cosine similarity → verdict	1 angka → asli/palsu

Tahap 1 — Raw Signature Image (gambar masuk)

- **Apa yang terjadi:** sistem menerima satu citra tanda tangan. Sumbernya bisa dari dataset (CEDAR / GPDS / lokal), hasil scan, foto, atau coretan yang digambar langsung pengguna di aplikasi web.
- **Kenapa penting:** gambar masuk sangat beragam — beda ukuran, beda resolusi, ada yang latarnya putih, ada yang transparan. Sistem harus siap menerima semuanya apa adanya.
- **Cara menjelaskan:** “Ini pintu masuknya. Apa pun sumbernya — hasil scan, foto HP, atau digambar langsung di web — semua lewat jalur yang sama.”

Tahap 2 — Preprocessing (penyeragaman gambar)

Tahap ini punya empat langkah kecil yang berurutan:

- **a. Ratakan latar ke putih.** Kalau gambar punya area transparan, area itu diisi putih. Kalau gambar sudah padat (opaque), dibiarkan apa adanya. Pengecekan ini penting: kalau gambar padat salah dianggap transparan, seluruh bidang bisa berubah jadi hitam — ini bug nyata yang sempat kami temukan dan perbaiki.
- **b. Auto-crop ke area tinta.** Sistem mencari kotak terkecil yang memuat seluruh goresan tinta, lalu memotong tepat di situ (dengan margin tipis). Ruang kosong di sekitar tanda tangan dibuang.
- **c. Taruh di kanvas persegi.** Hasil potongan ditempel di tengah kanvas berbentuk persegi supaya proporsi tanda tangan tidak gepeng saat diubah ukuran.
- **d. Resize 224×224 + normalisasi ImageNet.** Ukuran diseragamkan ke 224×224 piksel (ukuran masukan standar EfficientNet), lalu nilai tiap kanal warna dinormalisasi memakai statistik ImageNet — sama seperti saat backbone dilatih dulu.
- **Kenapa penting:** tahap ini membuang variasi yang TIDAK ada hubungannya dengan identitas penulis (ukuran kertas, posisi tanda tangan, banyaknya ruang kosong). Tanpa ini, model bisa terkecoh hanya karena satu tanda tangan difoto lebih besar atau lebih ke pojok.
- **Cara menjelaskan:** “Semua tanda tangan kami ‘seragamkan’ dulu: dipotong pas di tintanya, ditaruh di tengah, disamakan ukurannya. Jadi model menilai BENTUK goresannya, bukan kebetulan cara pengambilan gambarnya.”

Tahap 3 — Frozen EfficientNet-B0 Backbone (ekstraksi ciri)

- **Apa yang terjadi:** gambar 224×224 dilewatkan ke jaringan konvolusi EfficientNet-B0. Keluarannya diringkas lewat global average pooling menjadi satu vektor berisi 1.280 angka yang merangkum ciri visual gambar.
- **Arti “frozen”:** bobot backbone ini TIDAK diubah sama sekali selama pelatihan. Karena hasilnya selalu sama untuk gambar yang sama, ciri 1.280 angka ini kami hitung SEKALI lalu disimpan (cache) ke disk.

- **Kenapa penting:** backbone sudah terlatih mengenali pola visual umum (garis, lengkung, sudut, tekstur). Membekukannya memberi tiga keuntungan: hemat komputasi (cukup CPU, tanpa GPU), pengetahuan lamanya tidak rusak, dan kontribusi bagian yang kami latih bisa diukur terpisah.
- **Cara menjelaskan:** “Anggap ini ‘mata’ sistem — sudah terlatih, kami tidak ubah. Tugasnya mengubah gambar jadi 1.280 angka. Karena tidak berubah, cukup dihitung sekali lalu disimpan — itu sebabnya pelatihan kami bisa jalan tanpa GPU.”

Tahap 4 — Trainable Embedding Head (bagian yang dilatih)

- **Apa yang terjadi:** vektor 1.280 angka diringkas menjadi 512 angka melalui rangkaian: Linear (1280→512) → BatchNorm → ReLU → Dropout 0,3 → Linear (512→512) → BatchNorm, lalu dinormalisasi-L2.
- **Arti normalisasi-L2:** semua vektor dipaksa memiliki panjang 1, sehingga posisinya berada di permukaan sebuah “bola”. Akibatnya, membandingkan dua tanda tangan cukup dengan melihat SUDUT antar-vektor (cosine similarity), bukan panjangnya.
- **Kenapa penting:** inilah satu-satunya bagian yang kami latih (kurang dari 1 juta parameter). Tugasnya menata ulang ruang: tanda tangan orang yang sama dikumpulkan berdekatan, tanda tangan orang berbeda didorong menjauh.
- **Cara menjelaskan:** “Kalau backbone tadi ‘mata’, bagian ini ‘otak penilainya’. Hanya bagian kecil ini yang kami latih — dan ternyata itu sudah cukup untuk menaikkan seluruh metrik.”

Tahap 5 — Batch-Hard Triplet Loss (cara melatihnya)

- **Apa yang terjadi:** tiap putaran pelatihan mengambil 12 penulis × 4 gambar = 48 gambar. Untuk setiap gambar (disebut anchor), sistem mencari dua pembandingan di dalam batch itu:
- **Positif tersulit** — tanda tangan dari orang yang SAMA tetapi paling tidak mirip dengan anchor.
- **Negatif tersulit** — tanda tangan dari orang LAIN yang justru paling mirip dengan anchor.
- **Aturan yang dipaksakan:** jarak(anchor, negatif) harus lebih besar daripada jarak(anchor, positif) ditambah margin 0,3. Kalau belum, bobot head diperbaiki.
- **Kenapa harus “tersulit”:** kalau pembandingan diambil acak, sebagian besar pasangan sudah terlalu mudah dan model tidak belajar apa-apa. Dengan selalu memilih kasus tersulit, pelatihan tetap menantang sampai akhir.
- **Cara menjelaskan:** “Analoginya seperti latihan soal. Kalau terus dikasih soal gampang, kemampuan tidak naik. Jadi setiap putaran kami sengaja pilih kasus paling sulit: tanda tangan sendiri yang paling beda, dan tanda tangan orang lain yang paling mirip.”

Tahap 6 — Validation Setiap 150 Step (memilih model terbaik)

- **Apa yang terjadi:** pelatihan berjalan 1.500 langkah. Setiap 150 langkah (jadi 10 kali total), model diuji ke 33 penulis hold-out — penulis yang TIDAK pernah dipakai melatih. Diukur nilai AUC-nya, dan model hanya disimpan bila AUC-nya terbaik sejauh ini.
- **Kenapa penting:** model di akhir pelatihan belum tentu yang terbaik — bisa saja sudah mulai overfitting (hafal data latih). Dengan menguji berkala ke penulis asing dan menyimpan yang terbaik, kami mengambil model di titik puncaknya.
- **Cara menjelaskan:** “Kami tidak asal ambil model di akhir latihan. Setiap 150 langkah kami ujikan ke orang-orang yang belum pernah dilihat model, lalu simpan versi yang paling bagus.”

Tahap 7 — Held-out Evaluation (ujian akhir)

- **Apa yang terjadi:** model final diuji pada 33 penulis hold-out dalam DUA skenario terpisah:

- **Random impostor** — tanda tangan asli dibandingkan dengan tanda tangan ORANG LAIN. Ini serangan “asal beda”.
- **Skilled forgery** — tanda tangan asli dibandingkan dengan PEMALSUAN atas orang yang sama, yaitu tanda tangan yang sengaja dijiplak. Ini serangan yang serius.
- **Cara memutuskan:** kedua gambar diubah jadi embedding, lalu dihitung cosine similarity-nya. Bila skor $\geq$ ambang (threshold) $\rightarrow$ dinyatakan ASLI; bila di bawahnya $\rightarrow$ PALSU.
- **Hasilnya:** random impostor AUC 0,979 (EER 7,07%, akurasi 90,20%); skilled forgery AUC 0,871 (EER 20,20%, akurasi 79,49%).
- **Kenapa dipisah:** tingkat kesulitan keduanya jauh berbeda. Bila digabung jadi satu angka rata-rata, kelemahan sistem terhadap pemalsu yang benar-benar berlatih akan tertutupi. Memisahkannya membuat laporan jujur.
- **Cara menjelaskan:** “Ini ujian akhirnya. Dan kami sengaja pisahkan dua jenis serangan — karena kalau digabung, kelemahan terhadap pemalsu yang serius jadi tidak kelihatan.”

Antisipasi Pertanyaan Saat Presentasi

- **“Kenapa backbone-nya dibekukan, tidak dilatih semua?”** Karena keterbatasan perangkat (hanya CPU, tanpa GPU), dan karena kami ingin mengukur seberapa besar kontribusi bagian head saja. Ternyata melatih head saja sudah menaikkan seluruh metrik.
- **“Kenapa pakai cosine similarity, bukan klasifikasi biasa?”** Karena penulis baru bisa muncul kapan saja. Dengan mengukur kemiripan, penulis yang belum pernah dilihat model tetap bisa diverifikasi tanpa melatih ulang.
- **“Kenapa akurasi skilled forgery lebih rendah?”** Karena pemalsu sengaja meniru bentuk global tanda tangan — yang justru paling mudah ditangkap model. Pembeda halusny ada pada dinamika goresan, dan itu belum sepenuhnya tertangkap.
- **“Apakah data uji pernah dilihat model?”** Tidak. Pembagian dilakukan di tingkat PENULIS: 188 penulis untuk melatih, 33 penulis disisihkan penuh. Tidak ada satu penulis pun yang muncul di kedua sisi.